

SIJE 백엔드 과제

발주서 변경 승인 프로세스 구현

과제 개요

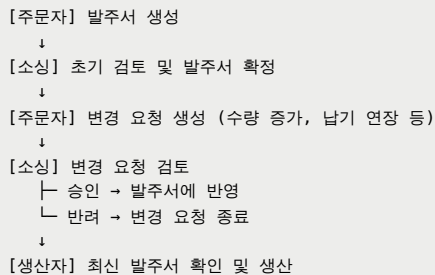
의류 생산 발주서의 주문자 변경 요청 → 소싱팀 승인 → 발주서 반영 프로세스를 구현합니다.

도메인 이해

역할 및 책임

- **주문자(Buyer)**: 발주서 내용 변경 요청 (수량, 납기일, 사양 등)
- **소싱팀(Sourcing)**: 변경 요청 검토 및 승인/반려
- **생산자(Manufacturer)**: 확정된 발주서 기준으로 생산 (읽기 전용)

프로세스 플로우



요구사항

1. 핵심 기능

발주서 생성 및 조회

- **발주서 생성**: 주문자가 상품명, 수량, 단가, 사양(색상, 사이즈 등), 납기일 등의 정보를 포함한 발주서를 생성합니다.
- **발주서 조회**: 발주서의 현재 상태를 조회할 수 있어야 합니다.

변경 요청 생성 (주문자)

주문자는 확정된 발주서에 대해 변경을 요청할 수 있습니다.

변경 요청 시 필요한 정보:

- 변경 사유
- 변경하려는 내용 (예: 수량을 1500벌로 변경, 납기일을 2025-03-25로 연장)

하나의 변경 요청에 여러 필드의 변경을 포함할 수 있습니다.

예시 시나리오:

- 기존 발주서: 티셔츠 1000벌, 납기일 2025-03-15
- 변경 요청: 수량 1500벌로 증가, 납기일 2025-03-25로 연장
- 승인 후: 발주서가 새로운 값으로 업데이트되며, 이전 상태(1000벌, 2025-03-15)도 조회 가능해야 함

변경 요청 승인/반려 (소싱팀)

소싱팀은 변경 요청을 검토하여:

- **승인:** 변경 내용을 발주서에 즉시 반영
- **반려:** 변경 요청을 거부하고 발주서는 기존 상태 유지

승인/반려 시 검토 의견을 함께 기록합니다.

변경 이력 조회 ★ 핵심 기능

다음 조회 기능들을 지원해야 합니다:

1. 변경 이력 조회

- 발주서에 승인된 모든 변경 이력을 시간순으로 조회
- 각 변경 시점의 발주서 전체 상태 또는 변경된 필드 정보를 확인할 수 있어야 함

2. 특정 버전 조회

- 발주서의 특정 버전(1번째, 2번째, 3번째 등) 상태를 조회
- 예: 2번째 버전 조회 시 → 당시의 수량, 납기일, 사양 등 모든 정보가 반환되어야 함

3. 특정 시점 조회

- 특정 날짜/시간 기준으로 당시의 발주서 상태를 조회
- 예: "2025-02-15 10:00:00 당시 이 발주서의 수량은?" → 해당 시점의 발주서 전체 상태 반환

4. 버전 간 비교

- 두 버전 사이에 어떤 필드가 어떻게 변경되었는지 비교
- 예: 버전1 vs 버전3 비교 → "수량: 1000 → 1500", "납기일: 2025-03-15 → 2025-03-25" 등의 차이 확인

시나리오 예시:

```
버전1 (2025-02-10 생성): 티셔츠 1000벌, 납기일 2025-03-15
버전2 (2025-02-15 변경): 티셔츠 1500벌, 납기일 2025-03-15
버전3 (2025-02-20 변경): 티셔츠 1500벌, 납기일 2025-03-25

조회 요구사항:
- 버전2 조회 → 1500벌, 2025-03-15 반환
- 2025-02-16 시점 조회 → 1500벌, 2025-03-15 반환 (버전2 상태)
- 버전1 vs 버전3 비교 → 수량 +500벌, 납기일 +10일 차이 확인
```

2. 비즈니스 규칙

변경 요청 생성 규칙

- 주문자만 변경 요청 생성 가능
- 발주서 상태가 **Confirmed** 이상일 때만 변경 요청 가능

- 동일 발주서에 대해 **Pending** 상태의 변경 요청이 있으면 신규 생성 불가
- 변경 항목은 1개 이상 필수

승인/반려 규칙

- 소싱팀만 승인/반려 처리 가능
- **Pending** 상태의 변경 요청만 처리 가능
- 승인 시 발주서에 즉시 반영
- 반려 시 발주서는 기존 상태 유지

데이터 일관성

- 변경 요청 승인 시 발주서 업데이트와 이력 저장은 하나의 트랜잭션으로 처리되어야 합니다
- 승인 처리 중 오류 발생 시 모든 변경사항이 롤백되어야 합니다

발주서 상태 관리

발주서는 다음 상태를 가집니다:

- **DRAFT (작성중)**: 주문자가 작성 중인 상태
- **PENDING (검토대기)**: 소싱팀 검토를 기다리는 상태
- **CONFIRMED (확정)**: 소싱팀이 확정된 상태 (이 상태부터 변경 요청 가능)
- **IN_PRODUCTION (생산중)**: 생산이 진행 중인 상태
- **COMPLETED (완료)**: 생산 및 납품이 완료된 상태

변경 요청 상태

변경 요청은 다음 상태를 가집니다:

- **PENDING (승인 대기)**: 소싱팀의 검토를 기다리는 상태
- **APPROVED (승인됨)**: 소싱팀이 승인하여 발주서에 반영된 상태
- **REJECTED (반려됨)**: 소싱팀이 반려한 상태

핵심 과제: 변경 이력 관리 전략

문제 상황

발주서는 생성 이후 여러 번 변경될 수 있으며, 다음 요구사항을 만족해야 합니다:

1. **과거 시점 조회**: "2월 15일 당시 이 발주서의 수량은 얼마였나?"
2. **변경 추적**: "누가, 언제, 무엇을, 왜 변경했는가?"
3. **비교 기능**: "초기 버전과 현재 버전의 차이는?"
4. **감사(Audit)**: 규제 준수를 위한 완전한 변경 이력 보존

당신의 과제

변경 이력을 관리하는 방법을 스스로 설계하고 구현하세요.

다음 질문들에 대해 깊이 고민하고, 선택한 방식과 그 이유를 설명해야 합니다:

고민해야 할 질문들

1. 무엇을 저장할 것인가?

- 변경 시점마다 전체 발주서 데이터를 저장할 것인가?
- 변경된 내용(델타)만 저장할 것인가?
- 둘 다 저장할 것인가?

2. 어떻게 저장할 것인가?

- 별도의 히스토리 테이블을 만들 것인가?
- 기존 테이블에 버전 필드를 추가할 것인가?
- JSON 형태로 저장할 것인가?
- 이벤트 스트림으로 관리할 것인가?

3. 조회는 어떻게 할 것인가?

- 특정 시점 조회 시 어떻게 데이터를 재구성할 것인가?
- 어떤 인덱스가 필요한가?
- 조회 속도를 어떻게 보장할 것인가?

제출물

1. 소스 코드

- NestJS 프로젝트
- 당신이 설계한 변경 이력 관리 방식 구현
- Entity/DTO 정의
- Service 로직
- Controller 구현
- 테스트 코드

2. 설계 문서 (DESIGN.md) ★ 필수

반드시 포함해야 할 내용:

```
# 변경 이력 관리 설계

## 1. 선택한 방식

### 개요
[당신이 선택한 방식을 명확히 설명]

### 데이터 구조
[ERD, 테이블 스키마, Entity 구조 등]

### 동작 방식
```

- 변경 승인 시 어떻게 저장되는가?
- 특정 시점 조회 시 어떻게 데이터를 가져오는가?
- 변경 비교는 어떻게 수행하는가?

2. 의사결정 과정

고려했던 대안들

[최소 2가지 이상의 다른 방식을 비교 설명]

최종 선택 이유

[구체적인 근거]

- 우리 도메인의 특성상...
- 요구사항 중 X가 가장 중요하므로...
- 구현 복잡도와 효과의 균형 측면에서...

3. 구현 상세

핵심 로직 설명

[주요 로직에 대한 설명과 코드 예시]

예외 상황 처리

[존재하지 않는 버전 조회, 데이터 불일치 등]

3. README.md

- 실행 방법
- 환경 설정 (Node.js, DB 등)
- API 명세
- 테스트 실행 방법

4. 테스트 시나리오

구현한 기능에 대한 테스트를 작성해야 합니다:

변경 저장 테스트

- 변경 요청 승인 시 이력이 올바르게 저장되는지 검증
- 여러 필드를 동시에 변경할 때 하나의 버전으로 저장되는지 검증

이력 조회 테스트

- 특정 버전(예: 2번째 버전)의 발주서를 정확히 조회할 수 있는지 검증
- 특정 시점(예: 2025-02-15 10:00:00)의 발주서 상태를 조회할 수 있는지 검증
- 존재하지 않는 버전 조회 시 적절한 에러를 반환하는지 검증

변경 비교 테스트

- 두 버전 간의 차이(어떤 필드가 어떻게 변경되었는지)를 정확히 조회할 수 있는지 검증

통합 시나리오 테스트

- 전체 플로우(생성 → 변경요청 → 승인 → 이력조회 → 비교)가 정상 동작하는지 검증
-

평가 범위 제외 사항

다음 사항들은 평가하지 않으므로 구현하지 않아도 됩니다:

- ❌ 대용량 데이터 처리 (수십만 건 이상)
- ❌ 동시성 제어 (낙관적/비관적 잠금)
- ❌ 분산 시스템 환경
- ❌ 실시간 성능 최적화
- ❌ 캐싱 전략

집중해야 할 것: 변경 이력 관리 방식의 설계와 선택 근거

기술 스택

필수

- NestJS
- TypeScript
- TypeORM 또는 Prisma (선택)
- PostgreSQL 또는 MySQL (선택)
- Jest (테스트)

선택 (가산 요소)

- class-validator (Validation)
 - @nestjs/swagger (API 문서화)
 - Docker (실행 환경)
-

제출

구현한 코드와 문서를 제출해주세요.

우리가 보고 싶은 것은:

1.  여러 방법을 알고 있는가
2.  각 방법의 장단점을 이해하는가
3.  상황에 맞는 선택을 할 수 있는가
4.  선택의 근거를 명확히 설명할 수 있는가